



Module : Apprentissage Automatique

Filière : BDIA'1 — Année universitaire 2025-2026

Chapitre 3

Classification

Prof. Dr Naoual Attaoui

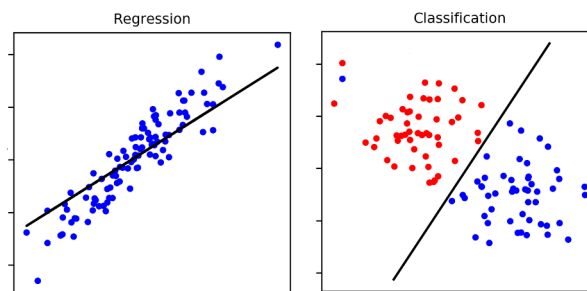
Table des matières

- 1. Introduction à la classification**
- 2. Formulation mathématique**
- 3. Régression logistique**
- 4. Fonction coût et apprentissage**
- 5. Métriques d'évaluation**
- 6. Algorithmes classiques de classification**
- 7. Prétraitement et validation**
- 8. Implémentation Python**

1. Introduction à la classification

La classification est l'un des problèmes fondamentaux de l'apprentissage automatique supervisé. Elle vise à modéliser la relation entre un ensemble de variables explicatives (appelées variables indépendantes ou *features*) et une variable cible discrète, représentant une classe ou une catégorie.

Contrairement à la régression, qui cherche à prédire une valeur numérique continue, la classification cherche à attribuer une observation à une catégorie déterminée. Par exemple, il peut s'agir de décider si un e-mail est un spam ou non, si une tumeur est bénigne ou maligne, ou encore si une image représente un chat, un chien ou un oiseau.



L'idée générale de la classification supervisée consiste à apprendre, à partir de données étiquetées, une règle permettant d'associer chaque observation à une classe. Selon l'algorithme utilisé, cette règle peut prendre la forme d'une frontière de décision explicite, d'un ensemble de règles, d'un calcul de probabilités ou d'une comparaison avec des exemples voisins. Le but reste de construire un modèle capable de distinguer correctement les différentes catégories.

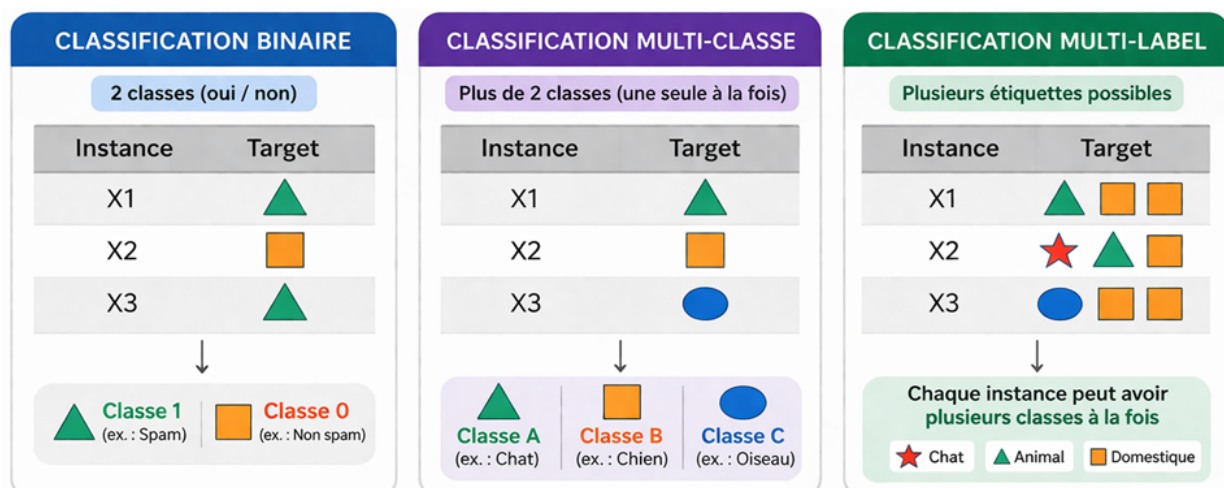
1.1. Types de classification

On distingue généralement plusieurs types de classification :

La **classification binaire** concerne les problèmes où il n'existe que deux classes, par exemple *oui/non*, *malade/sain* ou *fraude/non fraude*.

La **classification multi-classe** concerne les situations où une observation doit être affectée à une seule classe parmi plusieurs.

Enfin, la **classification multi-label** permet d'attribuer plusieurs étiquettes à une même observation.



1.2. Exemples d'applications

La classification est utilisée dans de nombreux domaines. En médecine, elle sert par exemple au diagnostic assisté. En cybersécurité, elle permet la détection de spam ou d'activités suspectes. En vision par ordinateur, elle intervient dans la reconnaissance d'objets. En finance, elle peut être utilisée pour la détection de fraude ou l'évaluation du risque.

1.3. Objectif de la classification

L'objectif principal d'un modèle de classification est de bien **généraliser**, c'est-à-dire de produire de bonnes prédictions sur de nouvelles données, et pas seulement sur les exemples utilisés pendant l'apprentissage. Un bon classifieur ne doit donc pas simplement mémoriser les données d'entraînement, mais apprendre une règle suffisamment fiable pour être réutilisée dans des situations nouvelles.

Ainsi, la classification répond à la question suivante : étant donné un ensemble de données d'entraînement, comment trouver la meilleure règle de décision capable d'attribuer correctement une classe à de nouvelles observations ?

✉ **Fil conducteur — Diagnostic médical** : Tout au long de ce chapitre, nous utiliserons le diagnostic de tumeurs mammaires comme exemple fil conducteur. À partir de mesures extraites d'images médicales (rayon, texture, périmètre, etc.), l'objectif est de prédire si une tumeur est bénigne (classe 1) ou maligne (classe 0).

2. Formulation mathématique

Dans un problème de classification, on considère un ensemble de données d'apprentissage composé de m observations. Chaque observation est décrite par n variables explicatives, appelées aussi **caractéristiques** ou **features**. L'ensemble des entrées est généralement représenté par une matrice :

$$X \in \mathbb{R}^{m \times n}$$

où m désigne le nombre d'exemples et n le nombre de variables.

À chaque observation est associée une sortie y , appelée **classe** ou **étiquette**. Dans le cas d'une classification binaire, cette variable cible prend généralement deux valeurs :

$$y \in \{0,1\}$$

Par exemple, $y=0$ peut représenter la classe *non spam* et $y=1$ la classe *spam*. Dans le cas d'une classification multi-classe, la sortie peut prendre plusieurs valeurs discrètes :

$$y \in \{1,2,\dots,K\}$$

où K est le nombre total de classes.

L'objectif de la classification est de construire une fonction de prédiction f capable d'associer à chaque observation x une classe probable :

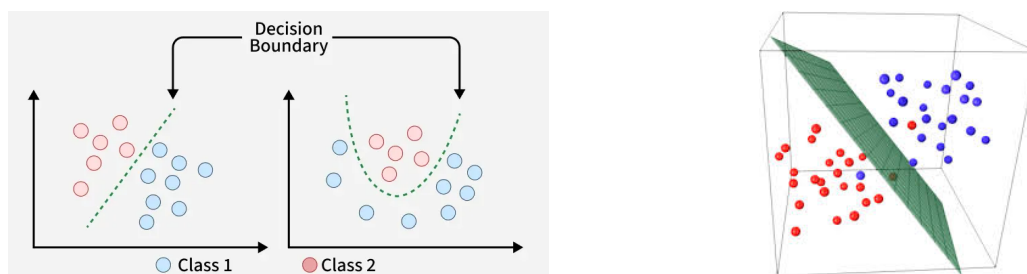
$$\hat{y} = f(x)$$

Dans certains modèles, cette fonction ne fournit pas directement une classe, mais un **score** ou une **probabilité** d'appartenance à une classe. La décision finale est ensuite obtenue en appliquant une règle de décision, par exemple un seuil.

Dans le cas binaire, on cherche souvent à estimer la probabilité conditionnelle : $P(y = 1 | x)$.

Si cette probabilité est supérieure à un seuil, souvent fixé à 0.5, l'observation est affectée à la classe 1 ; sinon, elle est affectée à la classe 0.

D'un point de vue géométrique, le problème de classification consiste à séparer les observations appartenant à des classes différentes dans l'espace des variables explicatives. Le modèle cherche donc à construire une **frontière de décision**. Dans les cas simples, cette frontière peut être une droite ou un plan. Dans des situations plus complexes, elle peut prendre une forme non linéaire.



Ainsi, la formulation mathématique de la classification repose sur trois éléments essentiels : les données d'entrée X , les classes cibles y , et la fonction de décision f qui permet de passer des observations aux classes prédites.

✉ **Fil conducteur — Diagnostic médical** : Chaque tumeur est décrite par $n = 30$ variables (rayon moyen, texture, périmètre, aire, etc.). La matrice X contient $m = 569$ observations et la cible $y \in \{0, 1\}$ indique si la tumeur est maligne (0) ou bénigne (1). Le modèle doit apprendre une fonction $f(x)$ capable de séparer ces deux classes.

3. Régression logistique

La régression logistique est l'un des modèles les plus utilisés en classification supervisée, en particulier pour les problèmes de **classification binaire**. Malgré son nom, il ne s'agit pas d'un modèle de régression au sens classique, car son objectif n'est pas de prédire une valeur continue, mais de déterminer la probabilité qu'une observation appartienne à une classe donnée.

3.1. Principe du modèle

L'idée de départ consiste à former une combinaison linéaire des variables explicatives :

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

ou, sous forme vectorielle :

$$z = w^T x + b$$

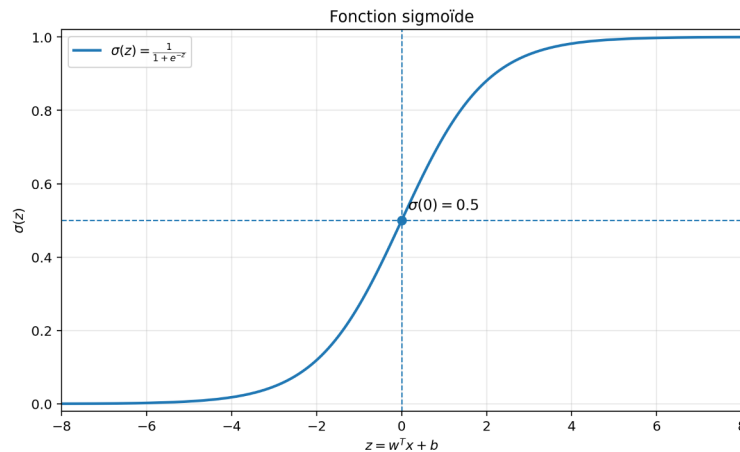
où w représente le vecteur des coefficients, x le vecteur des variables d'entrée, et b le biais.

Si l'on utilisait directement cette expression pour classer les données, le résultat pourrait prendre n'importe quelle valeur réelle, ce qui n'est pas adapté à une probabilité. Contrairement à la régression linéaire, la régression logistique est conçue pour produire une sortie comprise entre 0 et 1.

3.2. Fonction sigmoïde et interprétation probabiliste

Pour transformer z en une valeur comprise entre 0 et 1, la régression logistique applique une fonction de transformation appelée **fonction sigmoïde** :

$$\sigma(z) = 1 / (1 + e^{-z})$$



Cette fonction permet d'interpréter la sortie du modèle comme une probabilité :

$$P(y = 1 | x) = \sigma(w^T x + b)$$

Autrement dit, le modèle estime la probabilité que l'observation x appartienne à la classe 1. Plus cette probabilité est proche de 1, plus le modèle est confiant dans l'appartenance à cette classe. Inversement, plus elle est proche de 0, plus il estime que l'observation appartient à la classe 0.

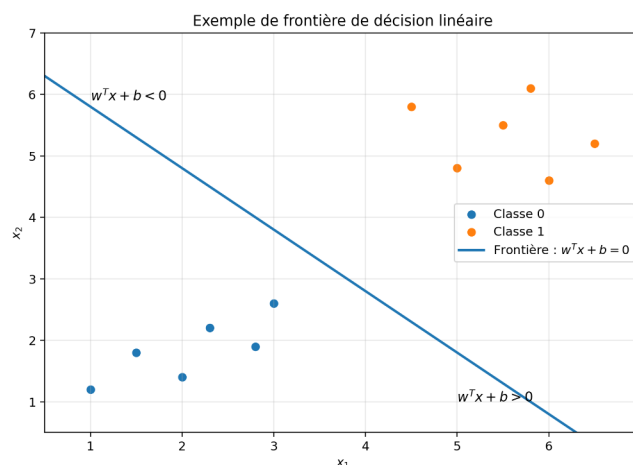
3.3. Règle de décision et frontière de décision

Pour prendre une décision finale, on applique généralement une règle de seuil. Le plus souvent, on choisit :

$$\hat{y} = \begin{cases} 1 & \text{si } P(y=1/x) \geq 0.5 \\ 0 & \text{sinon} \end{cases}$$

Ainsi, si la probabilité estimée dépasse 0.5, l'observation est classée dans la classe 1 ; sinon, elle est classée dans la classe 0.

D'un point de vue géométrique, la régression logistique définit une **frontière de décision** à partir de l'équation : $w^T x + b = 0$ qui correspond au seuil standard 0.5.



Cette frontière sépare l'espace des variables en deux régions correspondant aux deux classes. Lorsque cette séparation est suffisante, la régression logistique constitue un modèle simple, rapide et interprétable.

3.4. Intérêt de la régression logistique

La régression logistique présente plusieurs avantages. Elle est simple à mettre en œuvre, relativement facile à interpréter, et fournit directement des probabilités. C'est pourquoi elle est souvent utilisée comme premier modèle de référence en classification.

Elle constitue également une base importante pour comprendre des méthodes plus avancées. Les paramètres du modèle ne sont pas choisis arbitrairement : ils sont appris à partir des données en minimisant une fonction coût adaptée, que nous étudierons dans la section suivante.

✉ **Fil conducteur — Diagnostic médical** : La régression logistique calcule $z = w^T x + b$ à partir des 30 mesures de la tumeur, puis applique la sigmoïde pour obtenir $P(\text{bénigne} | x)$. Si cette probabilité dépasse 0,5, la tumeur est classée bénigne ; sinon, elle est classée maligne. La frontière de décision est un hyperplan dans l'espace à 30 dimensions.

4. Fonction coût et apprentissage

Pour entraîner un modèle de classification, il est nécessaire de mesurer l'écart entre les prédictions du modèle et les classes réelles. En régression logistique, la sortie du modèle est une probabilité comprise entre 0 et 1. Il faut donc utiliser une fonction coût adaptée à ce type de sortie.

Contrairement à la régression linéaire, où l'on utilise souvent l'erreur quadratique moyenne, la classification binaire utilise généralement la **fonction de coût logarithmique**, appelée aussi **entropie croisée** ou **log-loss** :

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

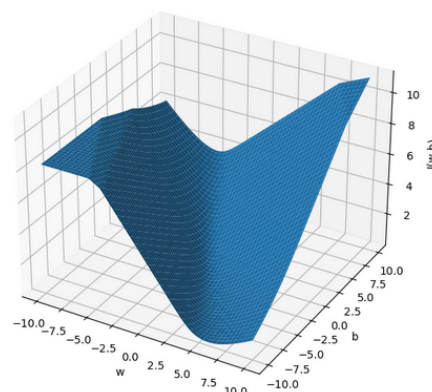
où :

- m est le nombre d'exemples d'apprentissage,
- $y^{(i)}$ est la classe réelle de l'exemple i ,
- $\hat{y}^{(i)}$ est la probabilité prédite par le modèle.

Cette fonction coût pénalise fortement les prédictions erronées faites avec une grande confiance. Par exemple, si la vraie classe est 1 mais que le modèle prédit une probabilité très proche de 0, la pénalité devient très importante. Cela pousse le modèle à produire des probabilités cohérentes avec les données.

L'apprentissage consiste alors à trouver les paramètres w et b qui minimisent cette fonction coût. Autrement dit, on cherche les coefficients qui rendent les prédictions du modèle aussi proches que possible des classes réelles.

Surface de la fonction de coût logistique



Pour réaliser cette minimisation, on utilise le plus souvent la **descente de gradient** (voir chapitre 2).

Au fil des itérations, la fonction coût diminue progressivement, jusqu'à atteindre une valeur minimale ou suffisamment faible. Le modèle apprend ainsi une frontière de décision qui sépare au mieux les deux classes.

✉ **Fil conducteur — Diagnostic médical** : Si le modèle prédit $P(\text{bénigne}) = 0,95$ pour une tumeur réellement maligne, la log-loss inflige une pénalité très élevée : $-\log(0,05) \approx 3,0$. Cette forte pénalisation pousse le modèle à ne jamais être confiant dans une mauvaise prédiction, ce qui est crucial en diagnostic médical.

5. Métriques d'évaluation

Une fois le modèle entraîné, il est nécessaire d'évaluer sa qualité. En classification, cette évaluation ne se limite pas au simple pourcentage de bonnes réponses, car certaines erreurs peuvent être plus importantes que d'autres selon le contexte. Par exemple, en diagnostic médical, classer un patient malade comme sain peut être bien plus grave que l'erreur inverse.

L'outil de base pour analyser les performances d'un classifieur est la **matrice de confusion**. Dans le cas binaire, elle permet de distinguer quatre situations : les **vrais positifs** (VP), les **vrais négatifs** (VN), les **faux positifs** (FP) et les **faux négatifs** (FN). Cette matrice donne une vision détaillée des prédictions correctes et des erreurs du modèle.

		Classes prédites	
		Prédit positif	Prédit négatif
Classes réelles	Réel positif	VP Vrai positif	FN Faux négatif
	Réel négatif	FP Faux positif	VN Vrai négatif

À partir de cette matrice, on définit plusieurs métriques importantes.

L'**accuracy** mesure la proportion totale de prédictions correctes :

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN}$$

Cette métrique est simple à comprendre, mais elle peut être trompeuse lorsque les classes sont déséquilibrées.

La **precision** mesure la proportion de vrais positifs parmi toutes les prédictions positives :

$$\text{Precision} = \frac{VP}{VP + FP}$$

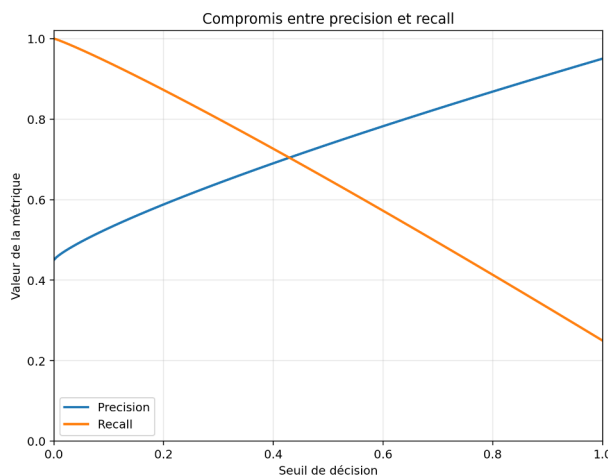
Elle indique donc à quel point une prédiction positive est fiable.

Le **recall**, ou sensibilité, mesure la proportion de vrais positifs correctement détectés parmi tous les positifs réels :

Cette métrique est particulièrement importante lorsque l'on veut éviter de manquer des cas positifs.

$$\text{Recall} = \frac{VP}{VP + FN}$$

Quand on augmente le seuil, le modèle devient plus exigeant pour déclarer un exemple positif. Les positifs prédits sont alors souvent plus sûrs, ce qui améliore la précision. En revanche, certains vrais positifs ne franchissent plus le seuil et sont ratés, donc le rappel diminue.



Le **F1-score** combine la précision et le rappel dans une seule mesure :

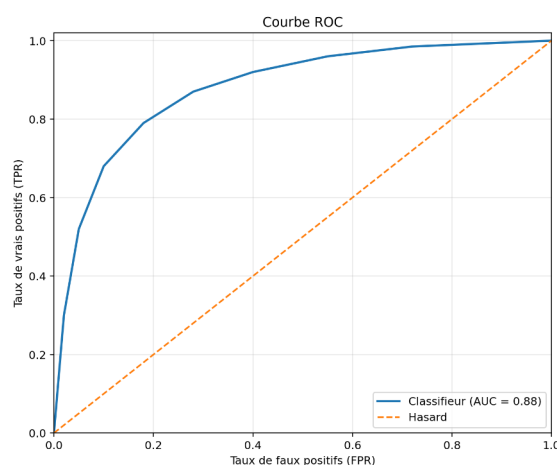
$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Il est utile lorsque l'on cherche un compromis entre ces deux grandeurs.

On peut aussi définir la **spécificité**, qui mesure la capacité du modèle à reconnaître correctement les négatifs :

$$\text{Spécificité} = \frac{VN}{VN + FP}$$

Enfin, pour évaluer le comportement du classifieur selon différents seuils de décision, on utilise souvent la **courbe ROC** (*Receiver Operating Characteristic*), qui représente le rappel en fonction du taux de faux positifs. Plus la courbe est proche du coin supérieur gauche, meilleur est le pouvoir de séparation du classifieur. L'**AUC** (*Area Under the Curve*) résume cette courbe par une seule valeur : plus elle est proche de 1, meilleur est le pouvoir de séparation du modèle.



Ainsi, le choix d'une métrique dépend toujours du problème étudié. Une bonne évaluation ne consiste pas seulement à mesurer combien de fois le modèle a raison, mais aussi à comprendre la nature et le coût de ses erreurs.

✉ **Fil conducteur — Diagnostic médical :** En diagnostic médical, un faux négatif (tumeur maligne classée bénigne) est bien plus grave qu'un faux positif. On privilégiera donc le recall pour minimiser les cas manqués, et le F1-score pour un compromis global. Sur notre dataset, le modèle atteint un AUC de 0,995, ce qui indique une excellente capacité de séparation.

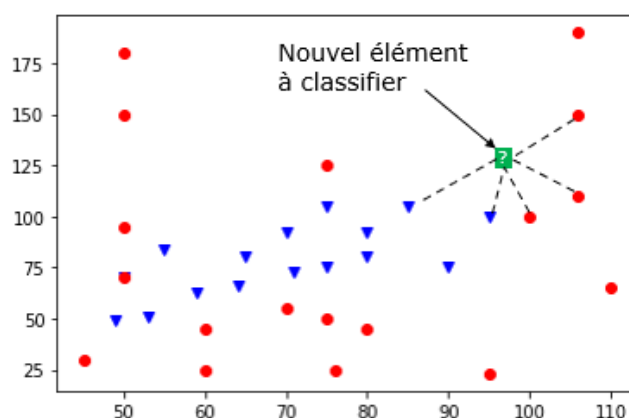
6. Algorithmes classiques de classification

En plus de la régression logistique, il existe plusieurs algorithmes classiques de classification, chacun reposant sur une idée différente. Le choix d'un algorithme dépend de la nature des données, de la complexité du problème, du besoin d'interprétation et du niveau de performance recherché.

6.1. k plus proches voisins (KNN)

Principe :

Le KNN attribue à une nouvelle observation la classe majoritaire parmi ses k voisins les plus proches dans l'espace des variables explicatives.



Idée mathématique essentielle :

L'algorithme repose sur le calcul d'une distance entre observations, souvent la distance euclidienne (mais d'autres distances comme Manhattan ou Hamming peuvent être choisies selon la nature des données) :

L'idée de base est que des observations proches ont tendance à appartenir à la même classe.

$$d(x, x^{(i)}) = \sqrt{\sum_{j=1}^n (x_j - x_j^{(i)})^2}$$

Avantages :

Le KNN est simple, intuitif et facile à mettre en œuvre. Il ne nécessite pas de phase d'apprentissage complexe. En effet, KNN possède une phase d'apprentissage très simple, qui consiste essentiellement à stocker les données d'entraînement. Il n'apprend pas de paramètres comme d'autres modèles. La véritable charge de calcul est déplacée vers la phase de prédiction, où il faut chercher les plus proches voisins ; c'est pourquoi on le classe parmi les méthodes d'apprentissage paresseux.

Limites :

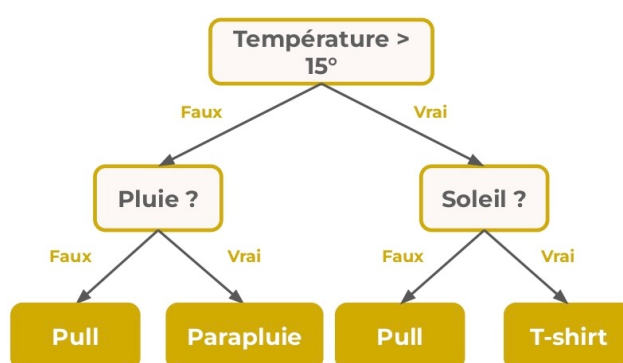
Il est sensible au choix de k , au bruit, ainsi qu'à l'échelle des variables. Il peut aussi devenir coûteux lorsque le nombre d'observations est grand.

Cas d'usage :

Il convient bien aux petits jeux de données et aux problèmes simples où la notion de voisinage est pertinente.

6.2. Arbres de décision**Principe :**

Un arbre de décision classe les observations à partir d'une suite de règles appliquées aux variables explicatives. Chaque nœud de l'arbre correspond à une question, et chaque feuille correspond à une décision finale.

**Idée mathématique essentielle :**

L'arbre cherche à réduire l'impureté des sous-ensembles obtenus après chaque séparation. On utilise par exemple l'indice de Gini :

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

ou l'entropie :

$$H = - \sum_{k=1}^K p_k \log_2(p_k)$$

où p_k est la proportion de la classe k dans le nœud.

Avantages :

Les arbres de décision sont faciles à comprendre, à visualiser et à interpréter.

Limites :

Ils peuvent surapprendre si l'arbre devient trop profond. Ils sont aussi sensibles aux variations des données.

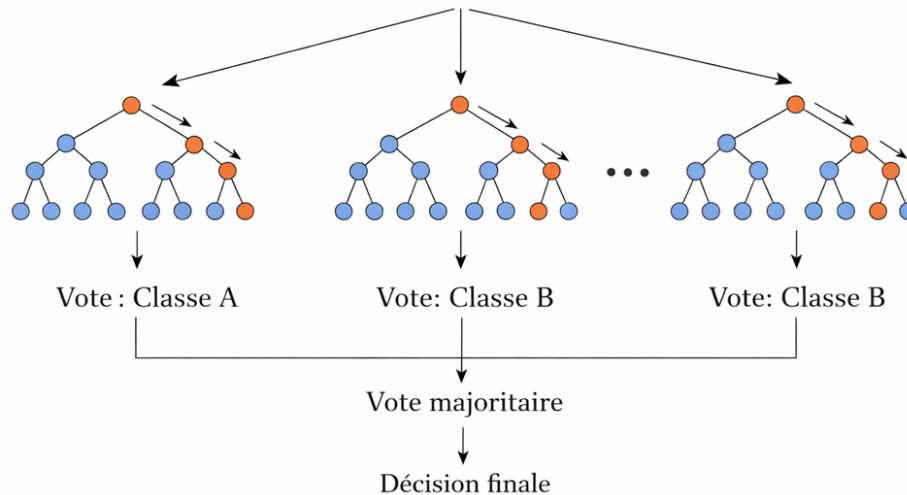
Cas d'usage :

Ils sont adaptés lorsque l'interprétabilité est importante et lorsque l'on souhaite expliquer clairement le mécanisme de décision.

6.3. Forêts aléatoires

Principe :

Les forêts aléatoires combinent plusieurs arbres de décision construits sur des sous-ensembles aléatoires des données et des variables. La classe finale est obtenue par vote majoritaire.



Idée mathématique essentielle :

La méthode repose sur l'idée qu'un ensemble de modèles combinés est souvent plus robuste qu'un seul modèle. La prédiction finale s'écrit conceptuellement comme un vote :

$$\hat{y} = \text{Classe la plus votée parmi les arbres}$$

Avantages :

Les forêts aléatoires sont robustes, performantes et moins sensibles au surapprentissage qu'un arbre unique.

Limites :

Elles sont moins interprétables qu'un arbre de décision seul et un peu plus coûteuses en calcul.

Cas d'usage :

Elles conviennent bien aux problèmes généraux de classification lorsque l'on cherche de bonnes performances sans réglage trop délicat.

6.4. Machines à vecteurs de support (SVM)

Principe :

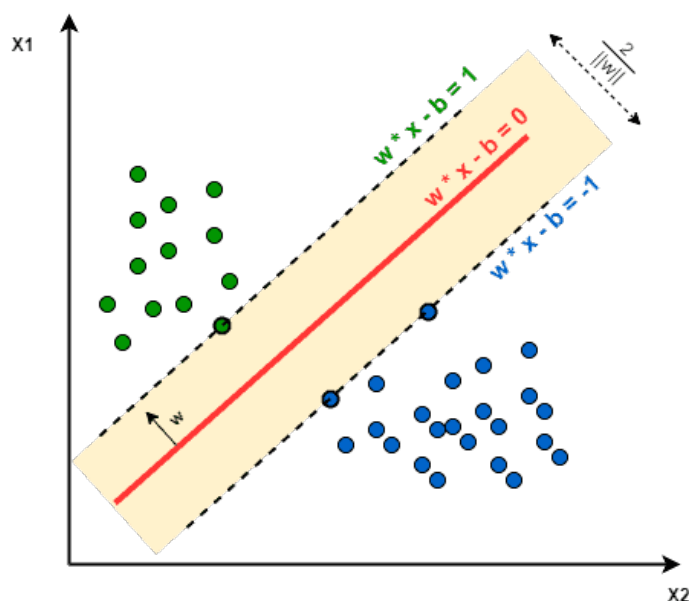
Le SVM cherche une frontière de décision qui sépare les classes avec la marge maximale.

Idée mathématique essentielle :

La frontière de décision est un hyperplan :

$$w^T x + b = 0$$

Le modèle cherche à maximiser la marge entre cet hyperplan et les observations les plus proches, appelées vecteurs de support.



Cela conduit à un problème d'optimisation de la forme :

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

sous contraintes de séparation.

Avantages :

Le SVM est souvent efficace lorsque les classes sont bien séparées. Il peut être très performant sur des jeux de données de petite ou moyenne taille.

Limites :

Il est moins intuitif que les arbres de décision, et son réglage peut être plus délicat. Il est aussi moins pratique sur de très grands jeux de données.

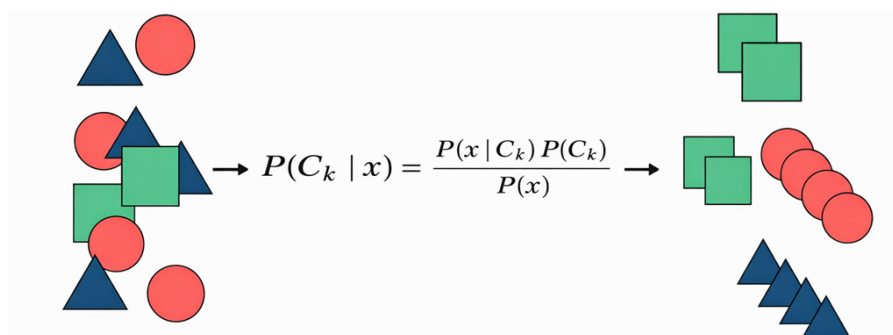
Cas d'usage :

Il est utile lorsque l'on cherche une séparation nette entre les classes et que le problème est de taille modérée.

6.5. Naive Bayes

Principe :

Naive Bayes classe une observation en calculant la probabilité qu'elle appartienne à chaque classe, puis en choisissant la plus probable.



Idée mathématique essentielle :

Il repose sur le théorème de Bayes :

$$P(C_k / x) = \frac{P(x / C_k) P(C_k)}{P(x)}$$

où :

$P(C_k|x)$: probabilité a posteriori. C'est la quantité qu'on veut calculer. On l'appelle probabilité a posteriori, car elle est calculée après avoir observé x .

$P(x|C_k)$: vraisemblance. C'est la probabilité d'observer x si on sait déjà que la classe est C_k . Autrement dit : si l'exemple appartient à la classe C_k , quelle est la probabilité d'observer ces caractéristiques x ?

$P(C_k)$: probabilité a priori. C'est la probabilité de la classe avant même de voir x . C'est notre connaissance initiale sur la fréquence de la classe.

$P(x)$: probabilité de l'observation. C'est la probabilité globale d'observer x , toutes classes confondues. Elle sert surtout de terme de normalisation pour que les probabilités finales somment bien à 1. En classification, on l'écrit souvent :

$$P(x) = \sum_j P(x | C_j) P(C_j)$$

avec l'hypothèse simplificatrice d'indépendance conditionnelle :

$$P(x_1, \dots, x_n / C_k) = \prod_{j=1}^n P(x_j / C_k)$$

Avantages :

Cet algorithme est rapide, simple et efficace dans plusieurs cas, notamment en classification de textes.

Limites :

Son hypothèse d'indépendance est souvent irréaliste, ce qui peut limiter ses performances.

Cas d'usage :

Il est particulièrement adapté aux problèmes de texte, de spam, ou aux situations où le nombre de variables est élevé.

6.6. Comparaison générale

Ces algorithmes reposent sur des idées mathématiques différentes. Le KNN classe par proximité, l'arbre de décision par réduction d'impureté, la forêt aléatoire par combinaison de plusieurs arbres, le SVM par marge maximale, et Naive Bayes par comparaison de probabilités.

Le choix de l'algorithme dépend donc du type de données, de la taille du problème, du besoin d'interprétation et des performances recherchées. En pratique, il est souvent utile de tester plusieurs méthodes avant de retenir celle qui convient le mieux au problème étudié.

✉ **Fil conducteur — Diagnostic médical** : Sur le dataset Breast Cancer, la régression logistique et le SVM obtiennent les meilleurs scores (accuracy $\approx 0,98$), tandis que l'arbre de décision est légèrement en retrait ($\approx 0,94$). Cependant, l'arbre offre l'avantage d'être facilement interprétable par un médecin, ce qui peut être décisif dans un contexte clinique.

7. Prétraitement et validation

Avant d'entraîner un modèle de classification, il est indispensable de préparer correctement les données. En pratique, la qualité d'un classifieur dépend non seulement de l'algorithme choisi, mais aussi de la qualité des données qui lui sont fournies. Le prétraitement et la validation constituent donc deux étapes essentielles de toute démarche d'apprentissage supervisé.

7.1. Importance du prétraitement

Les données réelles sont rarement parfaites. Elles peuvent contenir des valeurs manquantes, des erreurs de saisie, des variables qualitatives ou encore des variables numériques de grandeurs très différentes. Or, un algorithme de classification apprend directement à partir de ces données. Si elles sont mal préparées, les performances du modèle risquent de se dégrader.

Le prétraitement regroupe ainsi l'ensemble des opérations qui permettent de rendre les données cohérentes, exploitables et adaptées à l'apprentissage.

7.2. Traitement des données

Le prétraitement peut prendre plusieurs formes selon la nature du jeu de données. Il peut s'agir de traiter les valeurs manquantes, de corriger ou supprimer certaines données aberrantes, ou encore de sélectionner les variables réellement utiles à la tâche de classification.

Lorsque les variables sont qualitatives, il faut également les transformer en valeurs numériques afin qu'elles puissent être utilisées par les algorithmes. On utilise pour cela des techniques d'encodage, comme l'encodage par étiquettes ou l'encodage *one-hot*.

7.3. Mise à l'échelle des variables

Dans de nombreux problèmes, les variables numériques n'ont pas la même échelle. Par exemple, une variable peut varier entre 0 et 1, tandis qu'une autre varie entre 1 000 et 10 000. Cette différence peut perturber certains algorithmes, notamment ceux qui reposent sur des distances ou sur l'optimisation numérique.

Il est alors utile de normaliser ou de standardiser les données. La standardisation consiste à transformer une variable x selon :

$$x' = \frac{x - \mu}{\sigma}$$

où μ est la moyenne et σ l'écart-type. Cette transformation permet de centrer les données et de réduire les écarts d'échelle entre les variables.

7.4. Séparation des données

Une fois les données préparées, il faut organiser correctement l'apprentissage et l'évaluation. Pour cela, on sépare généralement les données en plusieurs ensembles :

- un **ensemble d'entraînement**, utilisé pour ajuster le modèle ;
- un **ensemble de validation**, utilisé pour choisir les hyperparamètres et comparer les modèles ;
- un **ensemble de test**, utilisé pour évaluer les performances finales sur des données jamais vues.

Cette séparation est essentielle, car elle permet de mesurer la capacité du modèle à généraliser.

7.5. Validation croisée

Lorsque le nombre de données est limité, une simple séparation entre apprentissage, validation et test peut donner une estimation instable des performances. On utilise alors la **validation croisée**, qui consiste à répéter plusieurs fois l'apprentissage et l'évaluation sur différentes partitions des données.

Dans la validation croisée en k blocs, les données sont divisées en k parties. À chaque itération, une partie est utilisée pour la validation et les autres pour l'entraînement. Cette méthode permet d'obtenir une évaluation plus robuste et plus fiable du modèle.

7.6. Déséquilibre des classes

Dans certains problèmes de classification, une classe peut être beaucoup plus représentée qu'une autre. Par exemple, dans un problème de détection de fraude, les cas normaux sont souvent largement majoritaires par rapport aux cas frauduleux.

Dans ce contexte, une métrique comme l'accuracy peut devenir trompeuse. Un modèle pourrait obtenir un très bon score global simplement en prédisant toujours la classe majoritaire. Il faut donc être particulièrement attentif au déséquilibre des classes, tant dans le prétraitement que dans l'évaluation.

7.7. Échantillonnage stratifié

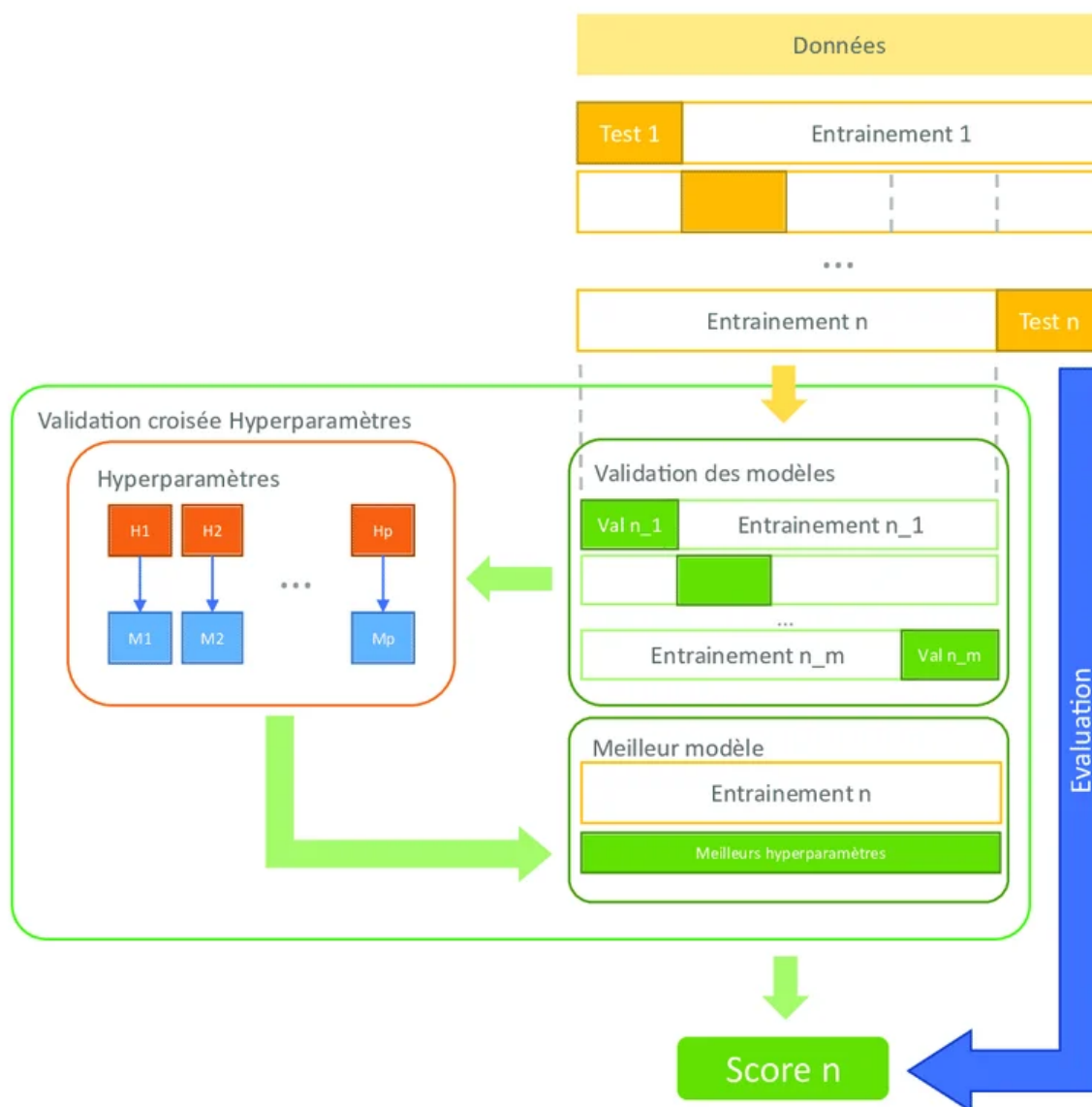
Lors de la séparation des données, il est souvent préférable d'utiliser un **échantillonnage stratifié**. Cela signifie que l'on conserve à peu près les mêmes proportions de classes dans les ensembles d'entraînement, de validation et de test.

Cette précaution est particulièrement importante lorsque les classes sont déséquilibrées, car elle permet d'obtenir des ensembles plus représentatifs du problème initial.

7.8. Choix des hyperparamètres

La validation ne sert pas seulement à estimer les performances d'un modèle. Elle permet aussi de comparer plusieurs réglages et de sélectionner les meilleurs hyperparamètres avant l'évaluation finale sur les données de test, comme le nombre de voisins dans KNN, la profondeur maximale d'un arbre de décision, ou le paramètre de régularisation d'un SVM.

Le choix de ces hyperparamètres influence directement la qualité du classifieur final. Une validation rigoureuse est donc indispensable pour sélectionner les meilleurs réglages.



7.9. Fuite de données et bonnes pratiques

Une erreur fréquente en apprentissage supervisé consiste à faire intervenir les données de test dans le prétraitement ou dans le choix du modèle. Ce phénomène est appelé **fuite de données** (*data leakage*). Il conduit à une évaluation artificiellement optimiste, car le modèle bénéficie indirectement d'informations qu'il ne devrait pas connaître.

Par exemple, il ne faut pas calculer les paramètres de standardisation sur l'ensemble complet des données avant la séparation. De même, il ne faut pas utiliser l'ensemble de test pour ajuster les hyperparamètres. En pratique, il est préférable d'organiser le prétraitement et l'apprentissage dans une même chaîne de traitement cohérente, souvent appelée *pipeline*, afin de limiter ce type d'erreur.

✉ **Fil conducteur — Diagnostic médical** : Dans notre dataset, les 30 variables ont des échelles très différentes (rayon moyen ≈ 14 mm, aire moyenne ≈ 650 mm²). Sans standardisation, un algorithme comme le KNN ou le SVM serait dominé par l'aire. La séparation stratifiée garantit également que les proportions bénigne/maligne (63 %/37 %) sont préservées dans chaque ensemble.

8. Implémentation Python

Dans cette section, nous illustrons la mise en œuvre d'un pipeline complet de classification supervisée : chargement des données, séparation en ensembles d'apprentissage et de test, standardisation, entraînement d'un modèle de régression logistique, puis évaluation à l'aide des métriques étudiées précédemment.

8.1. Implémentation d'un modèle de classification

Nous présentons ici une implémentation simple d'un modèle de classification binaire en Python à l'aide de **scikit-learn**. L'exemple suivant utilise une base de données classique, puis entraîne une **régression logistique** sur des données standardisées.

```
# =====
# 1) Importation des bibliothèques
# =====
# numpy : calcul numérique
# pandas : manipulation de tableaux de données
# sklearn.datasets : jeux de données intégrés
# sklearn.model_selection : séparation train/test
# sklearn.preprocessing : standardisation
# sklearn.linear_model : modèle de régression logistique

import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# =====
# 2) Chargement du jeu de données
# =====
# Ce dataset contient des mesures calculées à partir d'images
# de tumeurs mammaires. La cible est binaire :
# 0 -> classe maligne
# 1 -> classe bénigne
# Le problème est donc un problème de classification binaire.

data = load_breast_cancer()

# X contient les variables explicatives
# y contient les classes à prédire
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")

# Affichage d'informations utiles
print("Dimensions de X :", X.shape)
print("Classes présentes :", np.unique(y))
print("Noms des classes :", data.target_names)

# =====
# 3) Séparation en ensemble d'apprentissage et de test
# =====
# On réserve ici 20 % des données pour le test.
# Le paramètre stratify=y permet de conserver la proportion
# des classes dans les deux ensembles.
# random_state garantit la reproductibilité du résultat.

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Taille de l'ensemble d'apprentissage :", X_train.shape)
print("Taille de l'ensemble de test :", X_test.shape)
```

```
Dimensions de X : (569, 30)
Classes présentes : [0 1]
Noms des classes : ['malignant' 'benign']
Taille de l'ensemble d'apprentissage : (455, 30)
Taille de l'ensemble de test : (114, 30)
```

```
Premières prédictions : [0 1 0 1 0 1 1 0 0 0]
Premières probabilités : [0.000e+00 1.000e+00
6.400e-03 5.357e-01 0.000e+00 9.920e-01 1.000e+00
0.000e+00 1.000e-04 0.000e+00]
```

Ce code met en évidence plusieurs étapes essentielles. Les données sont d'abord séparées en ensembles d'apprentissage et de test. Ensuite, les variables sont standardisées, ce qui est particulièrement utile pour les modèles sensibles à l'échelle des données. Enfin, la régression logistique est entraînée, puis utilisée pour produire des prédictions de classes ainsi que des probabilités.

```
# =====
# 4) Standardisation des variables
# =====
# La régression logistique fonctionne souvent mieux lorsque
# les variables sont sur des échelles comparables.
# On calcule la moyenne et l'écart-type sur X_train seulement.
# Ensuite, on applique cette transformation à X_train et X_test.
# Cela évite la fuite de données.

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)# fit + transform sur train
X_test_scaled = scaler.transform(X_test)      # transform seulement sur test

# =====
# 5) Création et entraînement du modèle
# =====
# max_iter=1000 permet d'éviter un arrêt prématuré si
# l'algorithme a besoin de plus d'itérations pour converger.

model = LogisticRegression(max_iter=1000)

# Apprentissage des paramètres du modèle sur les données train
model.fit(X_train_scaled, y_train)

# =====
# 6) Prédiction
# =====
# predict(...) renvoie directement les classes prédites
# predict_proba(...) renvoie les probabilités associées
# Ici,[:, 1] correspond à la probabilité d'appartenir à la classe 1

y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[:, 1]

# Affichage de quelques résultats
print("\nPremières prédictions :", y_pred[:10])
print("Premières probabilités :", np.round(y_prob[:10], 4))
```

8.2. Validation croisée du modèle

Afin d'obtenir une estimation plus robuste des performances du classifieur, on peut utiliser une **validation croisée** sur l'ensemble d'apprentissage. Le principe consiste à diviser les données d'apprentissage en plusieurs blocs, puis à répéter l'entraînement et la validation plusieurs fois en changeant le bloc utilisé pour la validation. Les performances obtenues sont ensuite moyennées.

```
# =====
# 7) Validation croisée sur l'ensemble d'apprentissage
# =====
# Ici, on n'utilise pas encore l'ensemble de test.
# L'idée est d'estimer plus solidement la performance du modèle
# en répétant plusieurs entraînements/validations sur X_train.

from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline

# -----
# Pourquoi un pipeline ?
# -----
# Le pipeline garantit que la standardisation est recalculée
# correctement à l'intérieur de chaque bloc de validation croisée.
# Cela évite la fuite de données.
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])

# -----
# Validation croisée stratifiée en 5 blocs
# -----
```

Scores de validation croisée : [0.967 0.989 0.978
0.989 0.967]
Accuracy moyenne : 0.978
Écart-type : 0.0098

Dans le cas de la classification, on utilise souvent une validation croisée **stratifiée**, qui conserve à peu près les mêmes proportions de classes dans chaque bloc.

```
# StratifiedKFold conserve la proportion des classes dans chaque fold.
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# -----
# Calcul de l'accuracy sur chaque bloc
# -----
scores = cross_val_score(
    pipeline,
    X_train,
    y_train,
    cv=cv,
    scoring="accuracy"
)

print("Scores de validation croisée :", np.round(scores, 4))
print("Accuracy moyenne :", round(scores.mean(), 4))
print("Écart-type :", round(scores.std(), 4))
```

8.3. Évaluation du modèle

Une fois le modèle validé sur l'ensemble d'apprentissage, on peut l'entraîner sur toutes les données d'apprentissage, puis l'évaluer sur l'ensemble de test. Cette étape fournit une estimation finale de la performance du classifieur sur des données jamais vues pendant l'apprentissage ni pendant la validation.

Dans le cas de la classification, plusieurs métriques doivent être examinées. L'accuracy donne la proportion globale de bonnes prédictions, mais elle ne suffit pas toujours. La précision, le rappel, le F1-score et l'AUC permettent d'obtenir une évaluation plus fine, en particulier lorsque les classes sont déséquilibrées.

```
# =====
# 8) Entraînement final sur tout l'ensemble d'apprentissage
# =====
# Après la validation croisée, on réentraîne le pipeline
# sur l'ensemble complet d'apprentissage.
# Le jeu de test reste totalement à part jusqu'à cette étape.

pipeline.fit(X_train, y_train)

# =====
# 9) Prédictions sur le jeu de test
# =====
# predict(...) renvoie les classes prédites
# predict_proba(...) renvoie les probabilités associées
# à chaque classe ; ici, on récupère la probabilité de la classe 1

y_pred = pipeline.predict(X_test)
y_prob = pipeline.predict_proba(X_test)[: , 1]

# =====
# 10) Calcul des métriques d'évaluation
# =====
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_prob)
cm = confusion_matrix(y_test, y_pred)
```

```
===== Évaluation finale sur le jeu de test =====
Accuracy : 0.9825
Precision : 0.9861
Recall : 0.9861
F1-score : 0.9861
AUC : 0.9954
```

```
===== Matrice de confusion =====
[[41  1]
 [ 1 71]]
```

```
===== Rapport de classification =====
```

	precision	recall	f1-score	support
malignant	0.98	0.98	0.98	42
benign	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

Ce code met en évidence une idée importante : la validation croisée sert à estimer la qualité du modèle sur l'ensemble d'apprentissage, tandis que l'évaluation finale sur le jeu de test permet de mesurer sa capacité réelle de généralisation.

```
# =====
# 11) Affichage des résultats
# =====
print("==== Évaluation finale sur le jeu de test =====")
print("Accuracy :", round(acc, 4))
print("Precision :", round(prec, 4))
print("Recall    :", round(rec, 4))
print("F1-score  :", round(f1, 4))
print("AUC      :", round(auc, 4))

print("\n==== Matrice de confusion =====")
print(cm)

print("\n==== Rapport de classification =====")
print(classification_report(y_test, y_pred, target_names=data.target_names))
```

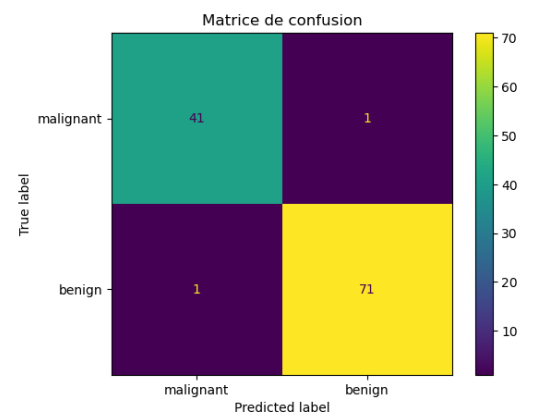
8.4. Visualisation de la matrice de confusion

Les métriques numériques donnent une information synthétique, mais une représentation visuelle de la matrice de confusion aide à mieux comprendre les erreurs du classifieur. Elle permet d'identifier directement les vrais positifs, les vrais négatifs, les faux positifs et les faux négatifs.

```
# =====
# 12) Visualisation de la matrice de confusion
# =====
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

# Affichage graphique de la matrice de confusion
ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    display_labels=data.target_names
)

plt.title("Matrice de confusion")
plt.show()
```



Cette figure est particulièrement utile pour interpréter les résultats dans un contexte applicatif. Par exemple, dans un problème médical, on cherchera à limiter au maximum les faux négatifs, car ils correspondent à des cas positifs non détectés.

8.5. Comparaison avec d'autres classifieurs

La régression logistique constitue un bon modèle de départ, mais il est souvent utile de la comparer à d'autres algorithmes de classification. Cette comparaison permet de montrer qu'il n'existe pas de méthode universellement meilleure : le choix dépend du problème étudié, de la structure des données, du besoin d'interprétation et des performances recherchées.

Dans l'exemple suivant, nous comparons la régression logistique avec KNN, un arbre de décision et un SVM.

```
# =====
# 13) Comparaison avec d'autres classifieurs
# =====
from sklearn.neighbors import KNeighborsClassifier
```

```
==== Comparaison de quelques classifieurs ====
Régression logistique -> Accuracy = 0.9825
KNN                  -> Accuracy = 0.9561
Arbre de décision    -> Accuracy = 0.9386
SVM                  -> Accuracy = 0.9825
```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline

models = {
    "Régression logistique": LogisticRegression(max_iter=1000),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Arbre de décision": DecisionTreeClassifier(max_depth=4,
        random_state=42),
    "SVM": SVC(probability=True)
}

print("==== Comparaison de quelques classifieurs ====")

for name, clf in models.items():
    # -----
    # Pour chaque modèle, on construit un pipeline
    # comprenant la standardisation puis le classifieur.
    # Cela garantit une procédure cohérente.
    # -----
    model_pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("model", clf)
    ])

    # Entraînement sur les données d'apprentissage
    model_pipeline.fit(X_train, y_train)

    # Prédictions sur le jeu de test
    y_pred_model = model_pipeline.predict(X_test)

    # Calcul d'une métrique simple pour comparaison
    acc_model = accuracy_score(y_test, y_pred_model)

    print(f"{name:22s} -> Accuracy = {acc_model:.4f}")

```

Ce type de comparaison peut être enrichi en utilisant d'autres métriques, comme le F1-score ou l'AUC. Il permet également d'ouvrir la discussion sur les avantages et les limites de chaque méthode : simplicité de la régression logistique, intuition du KNN, interprétabilité de l'arbre de décision, ou puissance du SVM.

8.6. Bilan

Cette section a montré comment mettre en œuvre en Python un pipeline complet de classification supervisée. La démarche suivie comprend plusieurs étapes essentielles : chargement des données, séparation en ensembles d'apprentissage et de test, standardisation, validation croisée, entraînement du modèle, évaluation finale, visualisation des erreurs et comparaison avec d'autres classifieurs. En pratique, cette démarche constitue la base de nombreux projets réels d'apprentissage automatique. D'autre part, elle montre qu'un bon pipeline de classification ne se limite pas au choix d'un algorithme : il repose aussi sur une préparation rigoureuse des données, une validation correcte et une interprétation pertinente des résultats.

✉ **Fil conducteur — Diagnostic médical :** Le pipeline complet appliqué au dataset Breast Cancer — standardisation, validation croisée stratifiée, régression logistique — atteint une accuracy de 0,98 et un AUC de 0,995. En comparant plusieurs classifieurs, on observe que la régression logistique et le SVM sont les plus performants sur ce problème, confirmant qu'un modèle simple bien calibré peut suffire en diagnostic médical.